



School of Computer Science and Applications
Annotix - Intelligent Data Annotation Platform
at
INFIPOSTS Pvt. Ltd.

An Internship Report submitted in partial fulfilment of the requirements for
the award of the Degree of Master of Computer Applications - MCA

Submitted by

Abrarali Sunasara

R24DE175

Under the guidance of

Internal Guide

Dr. Lokesh C K

Professor & Director

Computer Science & Application

External Guide

Mr. Manikant N

Founder & CEO

Infiposts Pvt.Ltd.

August 2026

Rukmini Knowledge Park, Kattigenahalli, Yelahanka, Bengaluru-560064

www.reva.edu.in



INFIPOSTS PVT LTD

CIN: U72900KA2021PTC153632
GSTIN: 29AAGCI3795K1ZV

Recognised by
#startupindia

Incubated at
REVA
NEST

Office Address: Reva Nest, Rukmini Knowledge Park, Yelahanka, Kattigenhalli,
Bengaluru, Karnataka 560064.

Date: 04th August 2026

To Whom It May Concern,

This is to certify that **Mr. Abrarali Zakirhusain Sunasara** is currently serving as a **Software Development Intern** at **Infiposts Private Limited**, from **10th February 2026 to 10th August 2026**, for a total internship duration of **6 months**. As of **04th August 2026**, he has successfully completed the internship responsibilities assigned to him to date and continues to remain an active intern with the organization for the remaining internship period.

During his internship, he has contributed to the development of **Annotix (Intelligent Data Annotation Platform)**, the **Binary App**, and the **HumanTask Platform**. His responsibilities have included developing backend APIs, designing database schemas and business logic, implementing authentication and role-based access control, supporting payment integration and task lifecycle management, structuring scalable backend architecture, coordinating backend requirements with frontend teams, establishing development workflows, optimizing APIs, resolving critical issues, and contributing to feature planning and production-ready implementations.

Throughout his internship, he has demonstrated sincerity, dedication, professionalism, and a strong willingness to learn. He has actively collaborated with cross-functional teams and shown a responsible approach toward his assigned work within a collaborative startup environment.

During his tenure with the organization to date, he has gained practical exposure to backend software development, API design, database management, scalable system architecture, collaborative engineering practices, and product development workflows. The internship experience has strengthened his technical knowledge as well as his analytical, communication, and problem-solving skills.

We appreciate his valuable contributions to date and confirm that his internship with **Infiposts Private Limited** is continuing as per the original internship schedule until **10th August 2026**. We wish him continued success in the remainder of his internship and in all his future endeavors.

For Infiposts Private Limited,

Manikanta.N
Founder & CEO



DECLARATION

I, **Abrarali Sunasara R24DE175** fourth semester student of Master of Computer Applications belonging to School of Computer Science and Applications, REVA University, declare that this work entitled “**Annotix: AI Data Annotation Management System**” is the result of the Internship done by me in Infiposts Pvt. Ltd. under the supervision of **Dr. Lokesh C K, Professor & Director, School of Computer Science & Application, REVA University, Bengaluru. Mr. Manikant N, Chief Executive Officer (CEO), Infiposts Pvt. Ltd.**

I am submitting this Internship report in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications by REVA University, Bengaluru- during the academic year 2025-26

Signed on:

Certified that this internship report submitted by <name of the candidate> has been carried out under our guidance and the declaration made by the candidates is true to the best of our knowledge.

Signature of the Internal Guide
Date:

Signature of the External Guide
Date:

Signature of the In-Charge Director
Date:

Signature of the Director
Date:

Official Seal of the School

TABLE OF CONTENTS

CHAPTERS	PAGE No.
1. INTRODUCTION	1
1.1 Introduction of the Internship	1
1.2 Objectives of the Internship	2
2. BRIEF PROFILE OF THE COMPANY	4
3. TECHNICAL DESCRIPTION	10
3.1 Technologies Involved	10
3.2 Challenges Faced	19
3.3 Role of the Intern	19
3.4 Professional Learning	26
4. CONCLUSION	28
5. APPENDIX – Sample Screenshots	29

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION OF THE INTERNSHIP

The industrial internship programme is an important part of the postgraduate curriculum, offering students the opportunity to apply academic knowledge in a professional software development environment. It helps bridge the gap between classroom learning and industry practices by exposing students to real-world software engineering, collaborative development, modern technologies, and enterprise application development. This report presents the work carried out during the internship at **Infipost Pvt. Ltd.**, a technology company specializing in cloud-based software solutions and AI-driven enterprise applications.

The internship was completed over a period of **twenty-four weeks (six months)**, during which the intern worked as a **Software Engineer Intern** as part of the product engineering team. Throughout this period, the primary focus was the design, development, and continuous enhancement of **Annotix**, an AI-powered Data Annotation Management Platform developed by Infipost Technologies. Annotix enables organizations to efficiently manage the complete annotation lifecycle by providing features such as project management, dataset handling, automated task generation, multi-level review workflows, user management, payment management, analytics, and real-time monitoring within a single cloud-based platform.

The internship involved contributing to multiple aspects of the platform rather than being limited to a single area of development. Responsibilities included designing and developing backend services using **Java Spring Boot**, implementing secure RESTful APIs, creating and optimizing PostgreSQL database schemas, integrating frontend and backend modules, implementing authentication and authorization using **JWT** and **Role-Based Access Control (RBAC)**, developing business logic for annotation workflows, and participating in feature implementation, debugging, performance optimization, testing, and system integration. Regular collaboration with frontend developers, QA engineers, UI/UX designers, and project managers provided valuable exposure to cross-functional product development in a professional engineering environment.

The development process followed the **Agile Software Development** methodology, with work organized into iterative sprints involving sprint planning, daily stand-up meetings, requirement discussions, code reviews, testing cycles, and continuous feature improvements. This workflow provided practical experience with the complete **Software Development Life Cycle (SDLC)**, including requirement analysis, system design, implementation, integration, testing, deployment, maintenance, and iterative product enhancement.

Working on Annotix provided an opportunity to gain practical experience in building enterprise-scale software that supports real-world AI data operations. The project required

implementing scalable system architecture, secure access control, workflow automation, efficient database design, API integration, and performance optimization while ensuring that multiple organizations, projects, datasets, annotators, reviewers, and administrators could operate simultaneously on the same platform. Exposure to these challenges helped develop both technical expertise and an understanding of how enterprise software is designed to meet reliability, scalability, and security requirements.

This report presents a detailed account of the internship experience, covering the objectives of the training, an overview of the organization, the technologies and tools used, the architecture and implementation of Annotix, the work carried out during the internship, challenges encountered and their solutions, knowledge and skills acquired, and the overall outcomes of the industrial training. Together, these sections demonstrate how the internship strengthened practical software engineering skills while providing valuable experience in the development of a modern cloud-based AI annotation management platform.

1.2 OBJECTIVES OF THE INTERNSHIP

The internship was carried out with the objective of gaining practical exposure to enterprise software development while applying academic knowledge to the design and implementation of a real-world application. The key objectives of the internship are as follows:

- To understand the complete software development lifecycle (SDLC) followed in a professional software development organization.
- To apply concepts of object-oriented programming, database management systems, software engineering, and web technologies in the development of a production-ready application.
- To gain practical experience in developing enterprise applications using the **Java Spring Boot** framework and related technologies.
- To design, develop, and integrate secure RESTful APIs for various functional modules of the **Annotix** platform.
- To implement secure authentication and authorization mechanisms using **JSON Web Tokens (JWT)** and **Role-Based Access Control (RBAC)**.
- To develop and optimize relational database schemas, entity relationships, and business logic using **PostgreSQL**, **Hibernate**, and **Spring Data JPA**.
- To understand modular software architecture and implement applications using layered architecture consisting of Controller, Service, Repository, and Entity layers.
- To participate in the development of key platform features, including project management, dataset management, task generation, annotation workflows, review processes, dashboard services, and payment management.
- To gain practical experience with collaborative software development practices using **Git**, **GitHub**, Agile methodologies, sprint planning, code reviews, and version control.

- To perform API testing, debugging, validation, and documentation using tools such as **Postman**, while ensuring application reliability and correctness.
- To understand industry best practices related to exception handling, input validation, logging, security, performance optimization, and maintainable code development.
- To strengthen professional skills such as communication, teamwork, problem-solving, analytical thinking, task management, and time management through continuous interaction with cross-functional development teams.
- To build practical knowledge and confidence in developing scalable, secure, and maintainable enterprise applications, thereby preparing for a professional career in software engineering.

CHAPTER 2

BRIEF PROFILE OF THE COMPANY

Infiposts Pvt. Ltd. is a software product and technology company focused on developing modern cloud-based applications, enterprise software solutions, and AI-driven platforms for businesses across various industries. The company specializes in building scalable, secure, and high-performance software systems that simplify business operations through automation, intelligent workflows, and digital transformation. By combining modern software engineering practices with cloud technologies, Infiposts delivers solutions that help organizations improve operational efficiency, data management, and overall productivity.

The company develops enterprise-grade applications that emphasize scalability, maintainability, and security while addressing real-world business challenges. Its engineering teams work on designing robust backend systems, intuitive web applications, workflow automation platforms, secure API services, and cloud-native architectures capable of supporting large numbers of users and high volumes of data. The organization follows modern development standards to ensure that its products remain reliable, extensible, and easy to maintain as business requirements evolve.

One of the company's flagship products is **Annotix**, an AI-powered Data Annotation Management Platform developed to simplify and automate the complete annotation lifecycle. The platform enables organizations to manage datasets, automate task generation, coordinate annotation workflows, perform multi-level quality reviews, monitor project progress, manage payments, and generate analytical reports through a centralized cloud-based application. Designed for machine learning teams, data operations professionals, and enterprise organizations, Annotix provides a secure and collaborative environment for managing large-scale annotation projects efficiently.

The primary technology stack used at Infiposts includes **Java Spring Boot** for backend development, **React.js** for frontend applications, and **PostgreSQL** for relational database management. The company also utilizes RESTful APIs, JWT-based authentication, Role-Based Access Control (RBAC), Git-based version control, and Agile software development practices to deliver production-ready enterprise applications. Emphasis is placed on modular architecture, clean code principles, performance optimization, secure software development, and continuous improvement throughout the product development lifecycle.

The engineering culture at **Infiposts Pvt. Ltd.** encourages collaboration, innovation, and knowledge sharing. Development teams work closely with product managers, frontend developers, backend engineers, quality assurance professionals, and designers throughout the Software Development Life Cycle (SDLC). Regular sprint planning, daily stand-up meetings, code reviews, testing cycles, and continuous feedback ensure that software is developed efficiently while maintaining high quality standards. This collaborative environment provides developers with valuable exposure to industry-standard development practices and modern enterprise software engineering methodologies.

Table 2.1: Company Profile Summary

Attribute	Details
Company Name	Infiposts Pvt. Ltd.
Industry	Software Product Development and Enterprise Technology Solutions
Primary Product	Annotix – AI Data Annotation Management Platform
Core Services	Enterprise Software Development, Cloud-Based Applications, AI Workflow Automation, RESTful API Development
Technology Stack	Java, Spring Boot, React.js, PostgreSQL, RESTful APIs, JWT Authentication
Development Methodology	Agile Software Development, Sprint-Based Delivery, Code Reviews
Target Users	Enterprises, AI/ML Teams, Data Operations Teams, Research Organizations
Engineering Practices	Modular Architecture, Secure Development, Scalable System Design, Continuous Integration and Collaboration
Work Environment	Collaborative, Cross-Functional Teams, Continuous Learning, Quality-Focused Development

2.1 Organisational Structure

The engineering organisation at Infipost Technologies is structured around product-oriented teams, each responsible for the end-to-end development and maintenance of a specific platform or module. A typical product team comprises a project lead or engineering manager, senior backend developers, backend and frontend developer interns, a quality assurance engineer, and, where applicable, a UI/UX designer. This structure allows for focused ownership of individual platforms while still enabling knowledge sharing and coordination across teams through periodic cross-team reviews and shared engineering standards.

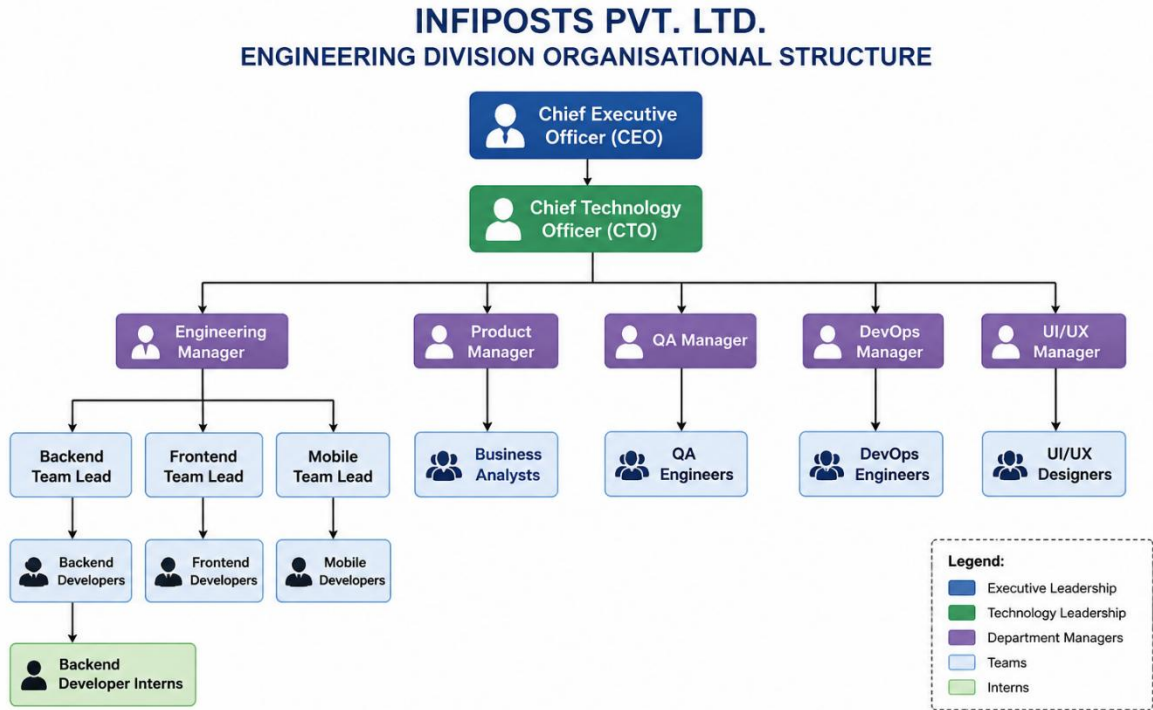


Figure 2.1: Organisational Structure of the Engineering Division at Infiposts Pvt. Ltd.

2.1.1 Vision and Mission

The vision of Infipost Technologies is to empower organisations to harness artificial intelligence and modern software engineering to solve practical business problems, by delivering platforms that are secure, scalable and easy to adopt. Its mission is centred on building high-quality backend systems and data-centric platforms that reduce operational friction for its clients, particularly organisations that rely on structured, high-quality data to train and evaluate machine learning models. This mission is reflected in the design priorities observed throughout the Annotix platform, where correctness, auditability and access control were treated as first-class engineering concerns rather than afterthoughts.

2.1.2 Company-wide Technology Stack

While individual product teams retain some flexibility in tool selection, Infipost Technologies maintains a set of standard technologies across its engineering organisation to ensure consistency, ease of cross-team collaboration, and simplified onboarding of new engineers and interns. Table 2.1(a) summarises this company-wide technology stack, within which the Annotix project was developed.

Table 2.1(a): Company-wide Technology Stack

Layer	Technologies Used
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Java, Spring Boot
Database	PostgreSQL, Hibernate, Spring Data JPA
Authentication & Security	Spring Security, JWT (JSON Web Token), Role-Based Access Control (RBAC)
API Communication	RESTful APIs, JSON
Build & Dependency Management	Maven
Version Control	Git, GitHub
API Testing	Postman
Development Methodology	Agile, Sprint-Based Development
Deployment Environment	Cloud-Based Deployment

2.1.3 Security and Data Handling Practices

Given that several of its platforms, including Annotix, handle sensitive project data and user credentials, Infipost Technologies places particular emphasis on secure development practices. These include enforced use of hashed and salted password storage, mandatory HTTPS for all externally facing endpoints, role-based restriction of sensitive operations, and peer review of any code that touches authentication, authorization or data-access logic. Interns and new engineers are introduced to these practices during onboarding, and adherence to them is verified through the standard code-review process rather than left to individual discretion.

2.2 Products and Platforms

Infipost Technologies develops and maintains a portfolio of internal and client-facing platforms, of which Annotix, the AI Data Annotation Management Platform, formed the primary focus of this internship. The table below summarises representative categories of products developed by the company, illustrating the breadth of its engineering activity.

Table 2.2: Representative Platform Categories at Infipost Technologies

Area	Description
AI Data Annotation Platform	Development of Annotix, a cloud-based platform for managing datasets, annotation workflows, quality review, and project monitoring for AI and Machine Learning applications.
Enterprise Software Solutions	Development of secure and scalable web applications that automate business processes and improve operational efficiency.
Backend Development & REST APIs	Design and implementation of RESTful APIs, business logic, authentication, authorization, and workflow management using Spring Boot.
Cloud-Based Application Development	Development of scalable cloud-hosted applications that support secure access, collaboration, and high availability.
Database Management	Design and maintenance of relational databases using PostgreSQL for efficient storage, retrieval, and management of application data.

2.2.1 Client and Stakeholder Engagement Model

Throughout the internship, development activities were carried out following an **Agile** workflow, where features and enhancements were implemented in iterative sprints. Progress was regularly reviewed through team discussions, sprint meetings, and internal demonstrations, allowing developers, project managers, and team members to evaluate completed functionality and identify areas for improvement. For the **Annotix** platform, continuous feedback during the development process contributed to refining core modules such as dataset management, annotation workflows, role-based access control, dashboard reporting, and overall system performance. This iterative development approach ensured that new features were integrated smoothly while maintaining the stability, scalability, and usability of the platform.

2.3 Work Environment and Engineering Practices

The development team at **Infiposts Pvt. Ltd.** follows an Agile approach to software development, where project work is organized into iterative development cycles. Development activities involve requirement discussions, task planning, feature implementation, testing, debugging, and regular progress reviews. Version control is maintained using **Git** and **GitHub**,

enabling collaborative development and efficient management of source code. Team members work closely to implement new features, resolve issues, and continuously improve the platform while maintaining code quality and application stability. This collaborative development environment provided valuable exposure to modern software engineering practices and formed the foundation for the internship activities presented in the subsequent chapters.

CHAPTER 3

TECHNICAL DESCRIPTION

3.1 TECHNOLOGIES INVOLVED

The development of Annotix required the use of a modern, production-grade Java backend technology stack. Each technology was selected for a specific engineering purpose, ranging from application development and persistence to security, build management and collaborative version control. The following subsections describe each of the principal technologies employed during the internship, followed by a consolidated summary table.

3.1.1 Java 21

Java 21 served as the primary programming language for backend development. As a long-term support (LTS) release, Java 21 provided access to modern language features such as record classes, pattern matching for switch expressions, and enhanced concurrency support through virtual threads, all of which contributed to writing more concise, readable and maintainable backend code. Java's strong static typing, mature ecosystem and extensive library support made it well suited to building a large, multi-module enterprise backend such as Annotix.

3.1.2 Spring Boot

Spring Boot was used as the core application framework for building the Annotix backend. It provided an opinionated, convention-over-configuration approach to application development, including embedded server support, auto-configuration of common components, and a modular starter-dependency system that significantly reduced boilerplate configuration. Spring Boot's dependency injection container was used extensively to manage the lifecycle of service, repository and controller components, promoting loose coupling and testability throughout the codebase.

3.1.3 Spring Security and JWT Authentication

Spring Security was employed to implement the application's authentication and authorization layer. A stateless authentication mechanism based on JSON Web Tokens (JWT) was designed, in which a signed token is issued to a user upon successful login and subsequently presented with each request to authenticate the caller without the need for server-side session state. Custom security filters were implemented to intercept incoming requests, validate the JWT signature and expiry, and populate the Spring Security context with the authenticated user's identity and granted authorities before the request reached the controller layer.

3.1.4 Role-Based Access Control (RBAC)

Building upon the authentication layer, a role-based access control model was implemented to govern what actions different categories of users could perform within

Annotix. Roles such as Administrator, Project Manager and Annotator were defined, each associated with a distinct set of permissions governing access to project creation, dataset management, task assignment and annotation submission endpoints. RBAC was enforced both at the method level, using security annotations on service and controller methods, and at the data level, by filtering query results according to the requesting user's role and project membership.

Table 3.1(d) presents the permission matrix that was implemented to govern access across the three roles defined within Annotix, and against which the intern's authorization logic was tested during development.

Table 3.1(d): RBAC Permission Matrix

Action	Administrator	Project Manager	Annotator
Create / Manage Users	Yes	No	No
Create Project	Yes	Yes	No
Upload Dataset	Yes	Yes	No
Assign Annotation Task	Yes	Yes (own projects)	No
View Assigned Tasks	Yes	Yes	Yes (own tasks only)
Submit Annotation	No	No	Yes
View Project Progress Reports	Yes	Yes (own projects)	No

This matrix was translated directly into method-level security annotations and service-layer membership checks, and served as the reference specification against which the authorization test cases described in Section 3.1.16 were designed.

3.1.5 Hibernate and Spring Data JPA

Hibernate, accessed through the Spring Data JPA abstraction, was used as the object-relational mapping (ORM) layer connecting the Java domain model to the underlying PostgreSQL database. Entity classes were mapped to relational tables using JPA annotations, with relationships between entities such as Users, Projects, Datasets and Annotation Tasks expressed through appropriate association mappings. Spring Data JPA repositories provided a declarative mechanism for defining data access methods, reducing the amount of hand-written SQL required while still allowing custom JPQL and native queries where necessary for more complex data retrieval scenarios.

3.1.6 PostgreSQL

PostgreSQL was used as the primary relational database management system for Annotix. Its support for advanced data types, strong transactional guarantees, and robust indexing capabilities made it well suited to the platform's requirements, which included maintaining referential integrity across interconnected entities such as users, roles, projects, datasets and

annotation tasks, as well as supporting efficient querying as data volumes grew with the addition of new annotation projects.

3.1.7 Maven

Maven was used as the build automation and dependency management tool for the project. The project's dependencies, including Spring Boot starters, security libraries, database drivers and utility libraries, were declared in the Project Object Model (pom.xml) file, allowing for consistent, reproducible builds across development environments and facilitating packaging of the application for deployment.

3.1.8 REST APIs and JSON

All communication between the Annotix backend and its client applications was conducted through RESTful APIs exchanging data in JSON format. Endpoints were designed following REST conventions, using appropriate HTTP verbs (GET, POST, PUT, DELETE) and status codes to represent resource operations, and consistent JSON response structures were adopted to simplify integration for frontend and third-party consumers of the API.

3.1.9 Lombok

Project Lombok was used to reduce boilerplate code in entity, data transfer object (DTO) and service classes. Annotations such as `@Getter`, `@Setter`, `@Builder` and `@AllArgsConstructor` were used to automatically generate commonly required methods at compile time, improving code readability and reducing the likelihood of errors introduced through manually written accessor and constructor code.

3.1.10 Git, GitHub and Postman

Git was used for distributed version control of the source code, with GitHub serving as the remote repository hosting platform. Development followed a feature-branch workflow, with changes proposed through pull requests and reviewed by senior engineers prior to being merged into the main branch. Postman was used extensively for manual and automated testing of REST API endpoints, allowing request payloads, headers and authentication tokens to be configured and responses to be inspected during development and debugging.

3.1.11 Docker, IntelliJ IDEA and VS Code

Docker was used, where applicable, to containerise the application and its supporting services such as the PostgreSQL database, enabling a consistent development and testing environment across the team. IntelliJ IDEA was used as the primary integrated development environment (IDE) for Java backend development, providing advanced code navigation, refactoring and debugging capabilities, while Visual Studio Code was used for lightweight editing tasks, configuration file management and documentation.

Table 3.1: Summary of Technologies, Tools and Languages Used

3.1.12 Data Transfer Objects (DTOs) and API Contracts

In addition to the core frameworks listed above, the design of Annotix made deliberate use of the Data Transfer Object (DTO) pattern to decouple the internal JPA entity model from the structure of data exposed through the REST API. Rather than serialising entity classes directly, dedicated request and response DTOs were defined for each operation, allowing the API contract to remain stable even as the underlying entity model evolved, and preventing sensitive or internal fields, such as password hashes or internal audit columns, from being inadvertently exposed to API consumers. Mapping between entities and DTOs was performed within the service layer, keeping this translation logic isolated from both the persistence layer and the presentation layer.

3.1.13 Logging and Monitoring

Structured logging was incorporated throughout the backend using SLF4J with the Logback implementation bundled with Spring Boot. Log statements were added at key points in the request-processing pipeline, including authentication attempts, service-layer business logic execution and exception handling, using appropriate log levels (INFO, WARN, ERROR) to distinguish routine operational events from conditions requiring attention. This logging infrastructure proved valuable both during development, for tracing the cause of defects, and as a foundation for future operational monitoring of the platform.

The overall backend of Annotix was structured using a layered architecture, separating concerns across distinct layers so as to promote maintainability, testability and scalability. Figure 3.1 illustrates this architecture.

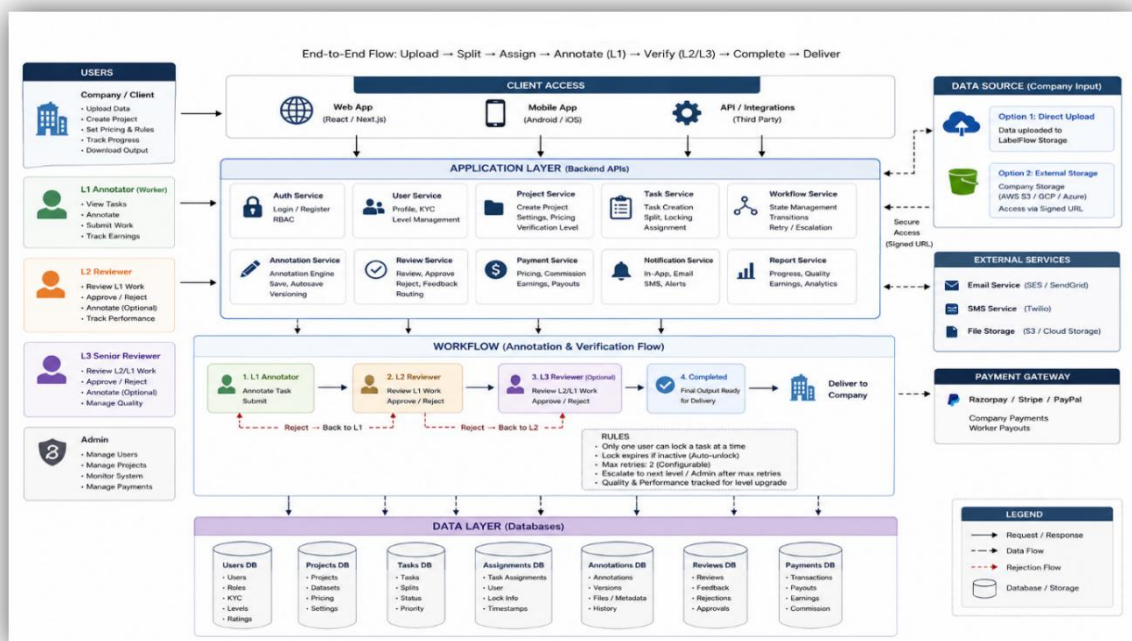


Figure 3.1: Layered architecture showing the flow of a request from the client through the Controller, Service, Repository and Database layers, with the Security filter chain intercepting each request.

As illustrated above, an incoming HTTP request first passes through the Spring Security filter chain, where the JWT token is validated and the caller's identity and role are established.

The request is then routed to the appropriate Controller, which delegates business logic to the Service layer. The Service layer applies validation, business rules and access-control checks before invoking the Repository layer, which interacts with the PostgreSQL database through Hibernate and Spring Data JPA. Responses flow back through the same layers, with DTOs used to shape the data returned to the client, ensuring that internal entity structures are not directly exposed through the API.

3.1.14 Database Design and Entity Relationships

The relational data model underlying Annotix was designed to capture the core domain concepts of the platform, namely users, roles, projects, datasets and annotation tasks, along with the relationships that connect them. Careful attention was given to normalisation, so as to avoid data redundancy, while still allowing efficient retrieval of related data through well-chosen indexes and join strategies. Table 3.1(a) below summarises the principal entities of the Annotix data model and their key relationships.

Table 3.1(a): Core Entities of the Annotix Data Model

Entity	Description	Relationship
User	Represents a registered user of the platform, including administrators, annotators, reviewers, and company users. Stores login credentials, profile details, and account information.	Many-to-One with Company; Many-to-One with Role; One-to-Many with Task; One-to-Many with Review
Role	Defines the permissions and access level assigned to users, such as Company Administrator, Annotator, Reviewer, or System Administrator.	One-to-Many with User
Company	Represents an organization using the Annotix platform and manages its users, projects, and datasets.	One-to-Many with User; One-to-Many with Project
Project	Represents an annotation project created by a company, containing project configuration, workflow settings, and datasets.	Many-to-One with Company; One-to-Many with Dataset
Dataset	Stores metadata for uploaded datasets and acts as the source for generating annotation tasks.	Many-to-One with Project; One-to-Many with Task
Task	Represents an individual annotation task generated from a dataset and assigned to an annotator for processing.	Many-to-One with Dataset; Many-to-One with User; One-to-One with Annotation

Entity	Description	Relationship
Annotation	Stores annotation results submitted by annotators, including annotation data, timestamps, and status.	One-to-One with Task; One-to-Many with Review
Review	Stores review decisions, comments, approval status, and quality verification performed by reviewers.	Many-to-One with Annotation; Many-to-One with User
Payment	Maintains payment records, earnings, payout history, and transaction details for completed annotation work.	Many-to-One with User; Many-to-One with Project
Audit Log	Records important user activities and system events such as logins, uploads, task assignments, reviews, and payment processing.	Many-to-One with User

This model allowed a single user to hold different responsibilities across multiple projects, for example acting as a Project Manager on one project while participating as an Annotator on another, a requirement that emerged from discussions with the product team during the design phase and that significantly influenced the decision to introduce a dedicated ProjectMembership entity rather than storing a single global role reference alone.

3.1.15 Representative API Endpoint Reference

To illustrate the structure of the REST API developed during the internship, Table 3.1(b) presents a representative subset of the endpoints implemented for the annotation task module, indicating the HTTP method, endpoint path, required role and purpose of each operation.

Table 3.1(b): Representative API Endpoints (Annotation Task Module)

Method	Endpoint	Required Role	Description
POST	/api/v1/auth/login	Public	Authenticates a user and returns a JWT access token.
POST	/api/v1/projects	Company Administrator	Creates a new annotation project with configurable settings.

Method	Endpoint	Required Role	Description
GET	/api/v1/projects/{id}	Authorized User	Retrieves project details and current project status.
POST	/api/v1/datasets/upload	Company Administrator	Uploads and registers a dataset for an annotation project.
POST	/api/v1/tasks/generate	Company Administrator	Automatically generates annotation tasks from the uploaded dataset.
GET	/api/v1/tasks/my-tasks	L1 Annotator	Retrieves annotation tasks assigned to the authenticated annotator.
PUT	/api/v1/tasks/{id}/submit	L1 Annotator	Submits completed annotation work for the review process.
PUT	/api/v1/reviews/{id}	L2 Reviewer / L3 Reviewer	Approves or rejects submitted annotations with review comments.
GET	/api/v1/dashboard/analytics	Company Administrator	Retrieves project statistics, progress reports, and analytical dashboard data.
GET	/api/v1/payments	Company Administrator	Retrieves payment details, worker earnings, and payout history.

Each endpoint listed above was accompanied by explicit request and response DTO definitions, standard HTTP status code usage (200 and 201 for successful operations, 400 for validation failures, 401 for unauthenticated requests, 403 for authorization failures, and 404 for missing resources), and corresponding entries in the internal API documentation maintained for the platform.

3.1.16 Testing Strategy

Testing of the Annotix backend was carried out primarily through manual and semi-automated API testing using Postman, supplemented by targeted unit testing of critical service-layer logic. For each module developed, a corresponding Postman collection was constructed covering positive test cases (valid requests expected to succeed), negative test cases (invalid input expected to be rejected with an appropriate validation error) and authorization test cases

(requests made with insufficient privileges, expected to be rejected with a 403 Forbidden response). Environment variables within Postman were used to manage JWT tokens for different user roles, allowing the same collection to be reused to verify RBAC enforcement across Administrator, Project Manager and Annotator accounts. Table 3.1(c) summarises the categories of test cases applied to each module.

Table 3.1(c): Testing Categories Applied Across Modules

Test Category	Purpose	Example
Functional Testing	Verify that each module performs its intended functionality correctly using valid inputs.	Creating a new annotation project with valid project details by a Company Administrator.
Validation Testing	Ensure the system correctly handles invalid or incomplete input by displaying appropriate validation messages.	Attempting to upload a dataset without selecting a supported file format.
Authentication Testing	Verify that only authenticated users can access protected resources using valid JWT tokens.	Accessing a protected API endpoint without logging in or with an expired JWT token.
Authorization (RBAC) Testing	Verify that users can access only the features permitted by their assigned roles.	An L1 Annotator attempting to create a new project or access administrative modules.
Integration Testing	Verify seamless communication between frontend, backend services, and the database.	Uploading a dataset through the frontend and confirming successful task generation in the backend.
Workflow Testing	Validate the complete annotation lifecycle from task creation to final approval.	An annotation moving successfully through L1 Annotation, L2 Review, L3 Review, and final approval.
Database Testing	Verify that application data is stored, updated, and retrieved correctly from PostgreSQL.	Confirming that annotation submissions, review comments, and task statuses are correctly persisted in the database.

Test Category	Purpose	Example
API Testing	Validate REST API functionality, request handling, response accuracy, and error handling.	Testing login, dataset upload, task submission, and dashboard APIs using Postman.
Performance Testing	Evaluate the platform's responsiveness and stability when handling multiple users and annotation tasks.	Multiple annotators simultaneously claiming and submitting annotation tasks without performance degradation.

3.1.17 Deployment and Environment Configuration

This attention to environment configuration proved particularly valuable during onboarding in the first month of the internship, when a consistent, reproducible local setup allowed development work to begin quickly without the delays that inconsistent manual environment configuration can otherwise introduce.

Figure 3.2 illustrates the Git branching and code review workflow followed by the team throughout the internship, which governed how the intern's contributions moved from local development to the shared main branch.

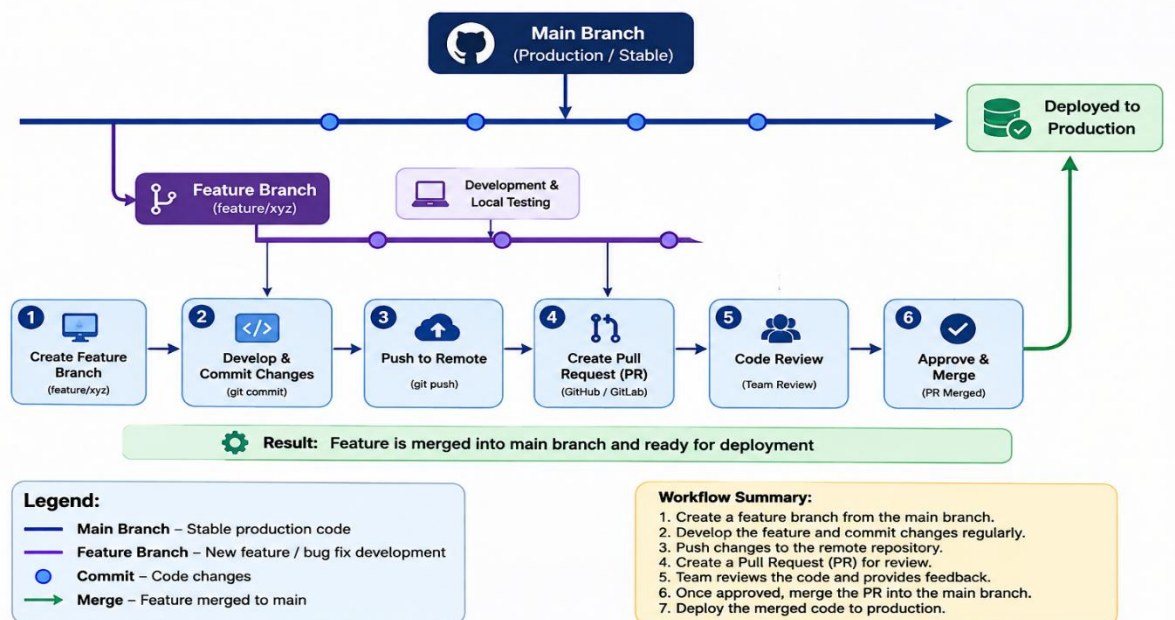


Figure 3.2: Git Branching and Pull Request Workflow

Figure 3.2: Feature-branch workflow showing progression from local development through pull request review to merge into the main branch.

Under this workflow, each task was developed on a dedicated feature branch created from the latest main branch. Once implementation and local testing were complete, the intern opened

a pull request describing the change, referencing the relevant task, and summarising how it had been tested. A senior developer would then review the pull request, leaving inline comments where changes were required; only once all review comments had been addressed and at least one approval had been granted was the branch merged into main, at which point the corresponding task card was moved to the Done column on the team's project board.

3.2 CHALLENGES FACED

As with any real-world software engineering assignment, the development of Annotix presented a number of technical challenges. Each challenge required careful analysis, research and iterative problem-solving, and the process of overcoming them contributed significantly to the intern's technical growth. The principal challenges encountered are described below.

3.2.1 Designing Scalable and Maintainable REST APIs

Designing an API surface that would remain consistent and extensible as new modules (projects, datasets, annotation tasks) were added required careful upfront planning of resource naming conventions, versioning strategy and response structures. This was addressed by adopting consistent RESTful conventions, standard DTO patterns and thorough API documentation prior to implementation.

This experience underscored that API design decisions made early in a project have long-lasting consequences, and that time invested in agreeing conventions before writing code is repaid many times over as the number of endpoints grows.

3.2.2 Implementing Secure JWT Authentication and Authorization

Correctly implementing stateless authentication, including token generation, signature validation, expiry handling and refresh-token considerations, required a solid understanding of the Spring Security filter chain. The custom authentication filter had to be carefully ordered within the filter chain to avoid interfering with other security configurations, and was resolved through incremental testing using Postman with valid, expired and malformed tokens.

Beyond the technical implementation, this challenge highlighted the importance of never storing plaintext credentials or embedding secrets directly in source code, and reinforced good practice around externalising configuration such as signing keys through environment-specific configuration files.

3.2.3 Managing Complex Entity Relationships in PostgreSQL

The domain model of Annotix involved multiple interrelated entities, including Users, Roles, Projects, Datasets and Annotation Tasks, connected through one-to-many and many-to-many relationships. Ensuring correct cascade behaviour, avoiding unintended cascading deletes, and preventing circular JSON serialisation issues required careful use of JPA annotations such as `@JsonManagedReference` and `@JsonBackReference`, along with deliberate fetch-type configuration.

Working through this challenge also improved understanding of the difference between eager and lazy fetching strategies in JPA, and of the trade-offs involved in choosing one over the other for different types of entity relationships.

3.2.4 Optimising Database Queries for Better Performance

As sample datasets grew during testing, certain list-retrieval endpoints exhibited the N+1 query problem, in which related entities were fetched through separate queries for each parent record. This was diagnosed using Hibernate's SQL logging and resolved through the use of JOIN FETCH queries and, where appropriate, projection-based DTO queries to retrieve only the required fields.

The resolution of this issue was verified by comparing the number of SQL statements generated before and after optimisation for representative list endpoints, confirming a substantial reduction in database round-trips.

3.2.5 Handling Validation and Global Exception Management

Ensuring that invalid input was rejected consistently and that meaningful error messages were returned to API consumers required the design of a centralised exception-handling mechanism. A global exception handler was implemented using `@ControllerAdvice`, mapping validation errors, resource-not-found conditions and access-denied exceptions to appropriate HTTP status codes and structured error responses.

A consistent error-response schema, comprising a status code, an error type and a human-readable message, was adopted across all modules so that frontend developers could handle errors predictably regardless of which endpoint produced them.

3.2.6 Debugging API Integration Issues

During integration with the frontend team, discrepancies occasionally arose between the expected and actual shape of API responses, particularly for nested resources. These issues were resolved through close collaboration with frontend developers, careful review of Postman collections, and refinement of DTO structures to precisely match documented API contracts.

These integration issues were ultimately reduced by adopting a practice of sharing sample request and response payloads with the frontend team before implementation began, rather than only after an endpoint was completed.

3.2.7 Maintaining Clean Architecture and Modular Code

Keeping business logic strictly within the service layer, avoiding logic leakage into controllers or repositories, and maintaining consistent package structure across an expanding codebase required ongoing discipline, reinforced through peer code review feedback.

Periodic self-review against the existing package structure, combined with constructive feedback received during code review, helped instil habits that were carried forward consistently into later modules developed during the internship.

3.2.8 Ensuring Secure Access through Role-Based Permissions

Correctly enforcing that, for example, an Annotator could not access administrative endpoints or modify projects to which they were not assigned, required careful design of method-level security checks combined with project-membership verification within the service layer, rather than relying on role checks alone.

Verifying this behaviour required deliberately writing negative test cases in Postman using tokens belonging to users with insufficient privileges, ensuring that access-control logic was validated as rigorously as the primary functional behaviour.

3.2.9 Managing Version Control during Collaborative Development

Working within a shared codebase alongside other developers occasionally resulted in merge conflicts, particularly in configuration and shared utility files. These were resolved through frequent branch synchronisation, smaller and more focused pull requests, and clear commit messaging.

Adopting a discipline of rebasing feature branches against the latest main branch before raising a pull request considerably reduced the frequency and severity of conflicts encountered later in the internship.

3.2.10 Improving Application Performance and Response Times

As the number of endpoints and the complexity of business logic grew, attention was given to reducing unnecessary database calls, introducing pagination for list endpoints, and profiling slow endpoints to identify and address performance bottlenecks.

Introducing pagination for endpoints returning potentially large result sets, in place of returning entire collections in a single response, was identified as a broadly applicable improvement adopted across multiple modules.

Taken together, the technologies, data model and architectural decisions summarised above formed the technical foundation upon which the intern's day-to-day development work, described in the remainder of this chapter, was carried out.

3.3 ROLE OF THE INTERN

Throughout the twenty-four-week internship, the intern was engaged as a Software Engineer - Backend Developer Intern within the team responsible for the Annotix platform. The role was primarily technical in nature, with the intern contributing directly to the design, implementation, testing and documentation of backend modules under the guidance of senior engineers. In addition to individual development tasks, the intern participated actively in the team's Agile ceremonies, including sprint planning, daily stand-up meetings, sprint reviews and retrospectives, and was expected to communicate progress, raise blockers and collaborate closely with both backend and frontend colleagues.

The intern's responsibilities evolved over the course of the internship, beginning with foundational tasks such as environment setup and codebase familiarisation, progressing through the independent development of complete backend modules, and culminating in contributions to performance optimisation, testing and documentation activities as the platform approached a stable release state. Table 3.2 summarises the broad areas of responsibility held by the intern.

Table 3.2: Areas of Responsibility

Area	Description of Responsibility
Feature Development	Contributed to the development and enhancement of multiple modules of the Annotix platform, including project, dataset, annotation, review, dashboard, and payment functionalities.
Backend & API Development	Developed and integrated RESTful APIs, implemented business logic, and ensured smooth communication between frontend and backend services.
Authentication & Security	Implemented secure authentication using JWT and enforced Role-Based Access Control (RBAC) across different user roles and platform modules.
Database Management	Designed, optimized, and maintained PostgreSQL database schemas, entity relationships, and data access layers using Hibernate and Spring Data JPA.
Workflow Implementation	Implemented and maintained annotation workflows, automatic task generation, multi-level review processes, and task management features.
Testing & Quality Assurance	Performed API testing, debugging, issue resolution, and functional verification using Postman while ensuring application stability and reliability.
Cross-Team Collaboration	Collaborated with frontend developers, QA engineers, and project managers for feature integration, bug resolution, and iterative product improvements.
Documentation & Maintenance	Maintained technical documentation, API references, implementation notes, and contributed to ongoing platform enhancements and system maintenance.

Team Structure and Communication: The intern operated within a product team comprising a project lead, two senior backend developers, one frontend developer, a quality assurance engineer, and the intern. Daily communication was conducted primarily through a team messaging platform and short daily stand-up calls, supplemented by longer synchronous sessions for sprint planning and sprint review held once every two weeks. Task tracking was

maintained through a shared project board, on which individual tasks were represented as cards moved through defined stages such as To Do, In Progress, In Review and Done, providing visibility to the entire team on the status of ongoing work at any given time.

This structure ensured that the intern's work was neither performed in isolation nor left unsupervised; rather, tasks were assigned with clear acceptance criteria, progress was visible to the wider team throughout development, and completed work was always subject to review before being considered final. This pattern of tight feedback loops, rather than long periods of unsupervised independent work, was identified by the intern as a significant contributor to the pace of skill development achieved over the course of the internship.

3.3.1 ACTIVITIES PERFORMED

The specific activities performed by the intern over the course of the internship are described below, followed by a month-wise summary of progress through the six-month engagement.

- Developed backend modules using Java and the Spring Boot framework, including the user management, project management, dataset management and annotation task management modules.
- Designed and implemented secure RESTful APIs following consistent naming conventions, resource structuring and HTTP status code usage.
- Implemented JWT-based authentication, including token generation upon login, token validation through a custom security filter, and secure password storage using industry-standard hashing.
- Developed a role-based access control (RBAC) system distinguishing between Administrator, Project Manager and Annotator roles, with appropriate method-level and data-level access restrictions.
- Designed and managed PostgreSQL database schemas, including definition of primary and foreign key relationships, indexes and constraints to preserve data integrity.
- Created entities, repositories, services and controllers following a consistent layered architecture across all backend modules.
- Implemented complete CRUD (Create, Read, Update, Delete) operations for users, projects, datasets and annotation tasks, along with supporting endpoints for task assignment and status tracking.
- Added request validation using Bean Validation annotations and implemented global exception handling to return consistent, informative error responses.
- Optimised database queries using Spring Data JPA, including the use of JOIN FETCH and projection queries to address performance issues identified during testing.
- Tested all developed API endpoints using Postman, including construction of test collections covering successful requests, validation failures and authorization failures.

- Debugged and resolved backend issues identified during development, integration testing and peer review.
- Integrated backend APIs with the frontend team, resolving discrepancies in request and response formats through direct collaboration.
- Used Git and GitHub for version control, including feature-branch development, pull requests and participation in merge-conflict resolution.
- Participated in code reviews, both as a contributor submitting code for review and as a reviewer providing feedback on colleagues' pull requests.
- Documented API endpoints, request and response formats, and backend workflows to support onboarding of new team members and future maintenance.

Table 3.3: Month-wise Summary of Internship Activities

Month	Phase	Key Activities
Month 1	Induction and Project Familiarization	Company onboarding, understanding the Annotix platform, development environment setup, technology stack familiarization, project architecture study, and codebase exploration.
Month 2	Core Module Development	Contributed to user management, authentication, project management, and dataset management modules while understanding the platform's workflow and business logic.
Month 3	Workflow & Feature Development	Implemented and enhanced annotation workflows, automatic task generation, task management, review process, and REST API integration across platform modules.
Month 4	Integration and Security	Integrated frontend and backend components, implemented JWT authentication, Role-Based Access Control (RBAC), optimized database operations, and improved API functionality.
Month 5	Testing and Optimization	Performed API testing using Postman, resolved bugs, optimized application performance, enhanced validation and exception handling, and supported feature refinement.
Month 6	Final Enhancement and Deployment Support	Implemented final feature improvements, resolved production issues, updated technical documentation, performed system testing, and contributed to overall platform stabilization before deployment.

The progression of activities described above followed a logical development sequence, in which foundational security and data modelling work in the earlier months enabled the more advanced feature and optimisation work undertaken in later months, reflecting the typical lifecycle of a growing backend platform.

3.4 PROFESSIONAL LEARNING

The internship provided substantial professional learning across both technical and interpersonal dimensions. On the technical front, the intern developed practical, hands-on proficiency in building production-grade backend systems using Spring Boot, moving beyond textbook exercises to engage with the realities of securing an application, designing a maintainable data model, and ensuring that code written by one engineer could be understood, reviewed and extended by others. Working with Spring Security and JWT provided a concrete understanding of how authentication and authorization are implemented in modern web applications, while the experience of designing PostgreSQL schemas and resolving query performance issues deepened understanding of relational database concepts learned in coursework.

Equally important was the exposure gained to professional software engineering practices that are not always emphasised in an academic setting. Participating in Agile ceremonies taught the value of breaking work into manageable, well-estimated units, of communicating progress transparently, and of surfacing blockers early rather than attempting to resolve every difficulty in isolation. The code review process, in particular, proved to be a significant source of learning, as feedback from senior engineers highlighted considerations such as naming clarity, separation of concerns, and edge-case handling that are not always apparent when writing code independently.

Table 3.4: Summary of Professional Learning

Category	Key Learning Outcomes
Technical Skills	Spring Boot application development, REST API design, JWT authentication, RBAC, JPA/Hibernate, PostgreSQL schema design, query optimisation
Software Engineering Practices	Layered architecture, exception handling, input validation, API documentation, Git-based collaborative workflows
Agile Methodology	Sprint planning, stand-up participation, task estimation, sprint review and retrospective processes
Debugging and Testing	Systematic use of Postman for API testing, log-based debugging, root-cause analysis of performance issues
Soft Skills	Communication, teamwork, time management, giving and receiving constructive feedback
Industry Awareness	Understanding of how security, scalability and maintainability considerations shape real-world engineering decisions

3.4.1 Self-Assessment of Strengths and Areas for Improvement

In reflecting on the internship experience, certain personal strengths were reinforced, while areas requiring further development were also identified. Among the strengths reinforced during the internship were a methodical approach to debugging, a willingness to ask clarifying questions rather than proceeding on assumptions, and steady improvement in the ability to design clean, layered code from the outset rather than refactoring extensively after the fact. Areas identified for continued improvement included the need for greater initial familiarity with large, pre-existing codebases, faster recognition of common architectural patterns already in use within a project, and further development of confidence in independently proposing design alternatives during technical discussions rather than primarily implementing solutions suggested by senior engineers.

This self-assessment, discussed informally with the intern's mentor towards the end of the internship, is considered a valuable outcome in its own right, as it provides a concrete basis for continued skill development beyond the conclusion of the internship, whether in subsequent academic coursework, personal projects, or future professional roles.

3.4.2 Application of Academic Knowledge

The internship provided an excellent opportunity to apply the theoretical concepts learned during the MCA programme to the development of a real-world software product. Working on **Annotix** helped bridge the gap between classroom learning and professional software engineering by allowing academic knowledge to be implemented in a practical development environment.

Concepts from **Database Management Systems** were applied while designing relational database structures, defining entity relationships, and managing data using PostgreSQL and Hibernate. Similarly, principles of **Object-Oriented Programming** and **Software Engineering**, such as modular design, layered architecture, abstraction, and code reusability, played an important role in developing maintainable and scalable application modules.

Knowledge gained from **Web Technologies** and **Computer Networks** was useful while designing and integrating RESTful APIs, understanding client-server communication, and handling HTTP requests and responses. The internship also strengthened understanding of application security through the implementation of **JWT-based authentication** and **Role-Based Access Control (RBAC)**, demonstrating how secure access control mechanisms are applied in enterprise applications..

Overall, the internship reinforced the practical relevance of academic coursework while providing hands-on experience in developing a scalable enterprise application like **Annotix**, significantly improving technical knowledge, software development practices, and readiness for a professional career in software engineering.

CHAPTER 4

CONCLUSION

The six-month internship undertaken at Infipost Technologies Pvt. Ltd. as a Software Engineer - Backend Developer Intern proved to be a comprehensive and formative professional experience. Working on Annotix, an AI Data Annotation Management Platform, provided the opportunity to apply theoretical knowledge of programming, database systems and software engineering principles to the design and development of a real, production-oriented backend system. Over the course of the internship, the intern progressed from foundational orientation and environment setup to the independent design and implementation of complete backend modules, including secure authentication, role-based access control, project and dataset management, and annotation task tracking, all built upon a layered Spring Boot architecture backed by a PostgreSQL database.

The objectives established at the outset of the internship, as detailed in Chapter 1, were substantially achieved. Practical exposure to a professional development environment was gained through active participation in Agile sprints, code reviews and cross-team collaboration. Proficiency was developed in the core technologies underpinning the Annotix platform, including Java 21, Spring Boot, Spring Security, Hibernate, Spring Data JPA and PostgreSQL, and a solid working understanding was gained of how authentication, authorization and data validation are implemented securely in enterprise applications. The challenges encountered during the internship, ranging from the design of scalable REST APIs to the optimisation of database queries, provided valuable opportunities for problem-solving and reinforced the importance of careful architectural planning in software systems that are expected to grow in complexity over time.

Beyond the technical outcomes, the internship contributed significantly to the intern's professional development. Exposure to industry-standard engineering practices, including version control workflows, structured code review and Agile project management, provided insight into how software teams operate beyond the boundaries of individual coding tasks. The experience of collaborating with developers, testers and project leads strengthened communication and teamwork skills, while the responsibility of contributing to a live platform used within the organisation fostered a strong sense of ownership and accountability for the quality and correctness of delivered work.

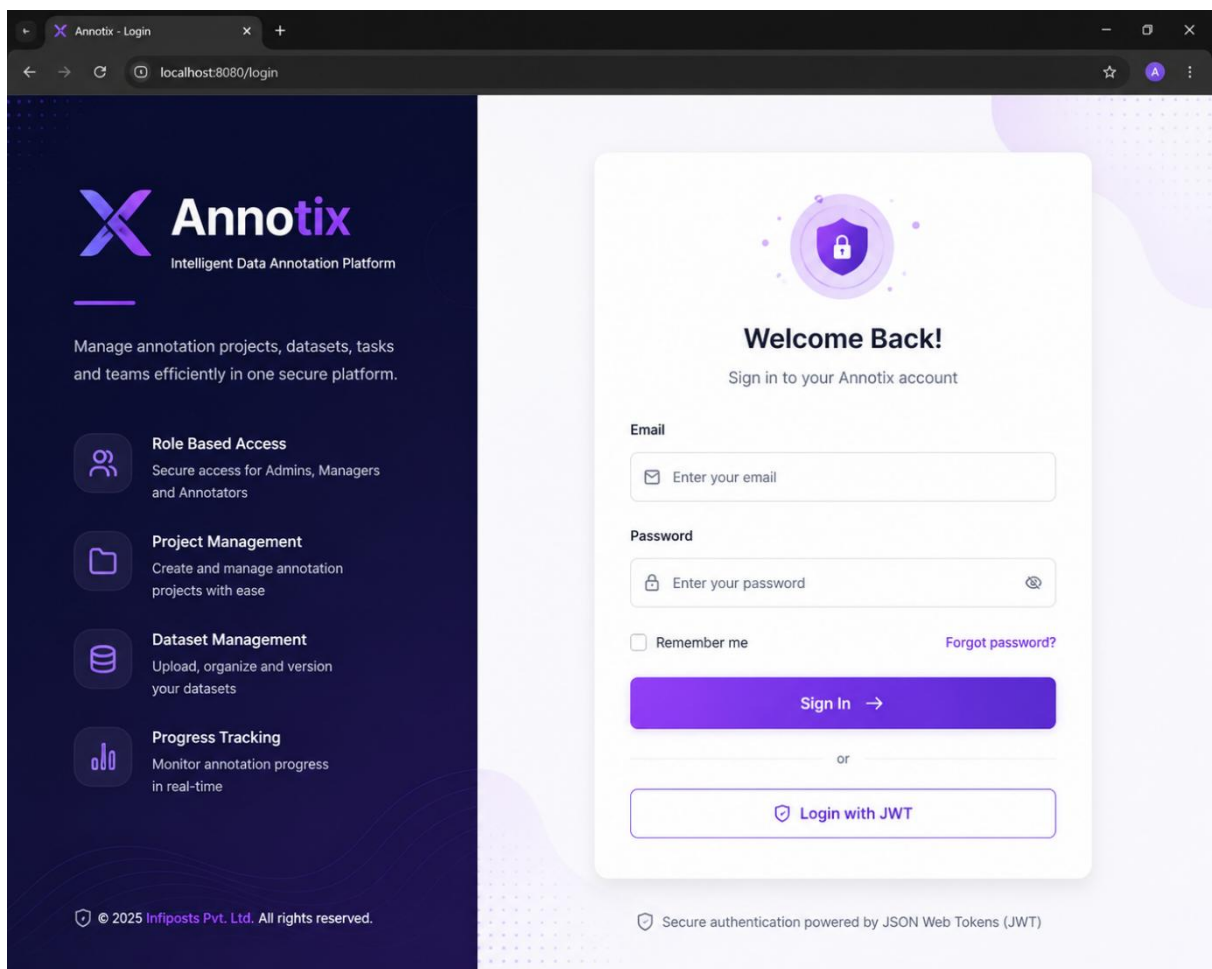
In conclusion, the internship at Infipost Pvt. Ltd. served as a valuable extension of academic learning into a professional context, equipping the intern with practical technical skills in backend software engineering as well as transferable soft skills applicable to any future professional role. The knowledge and experience gained through the development of the Annotix platform are expected to provide a strong foundation for future academic pursuits and for a career in software engineering, and the intern remains grateful for the guidance, mentorship and opportunities for growth provided by the organisation throughout the course of the internship.

CHAPTER 5

APPENDIX - SAMPLE SCREENSHOTS

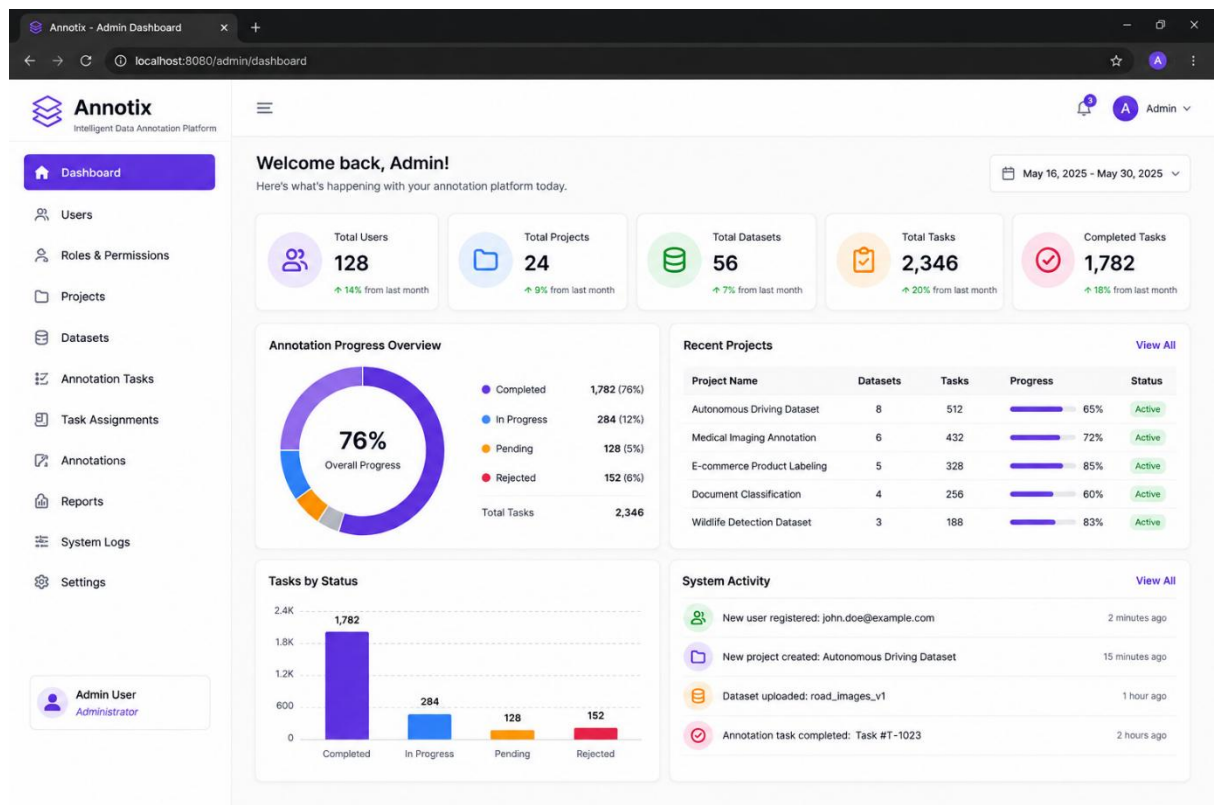
This appendix presents representative screenshots illustrating key screens and workflows of the Annotix platform as developed during the internship. Each figure is accompanied by a brief description of the functionality depicted, the backend module it corresponds to, and its relevance to the overall platform. As the intern's contributions were concentrated on the backend, the interfaces shown here reflect the frontend surfaces through which the backend APIs and business logic developed during the internship were ultimately exercised and validated.

Screenshot 5.1 Login Page with JWT-based Authentication



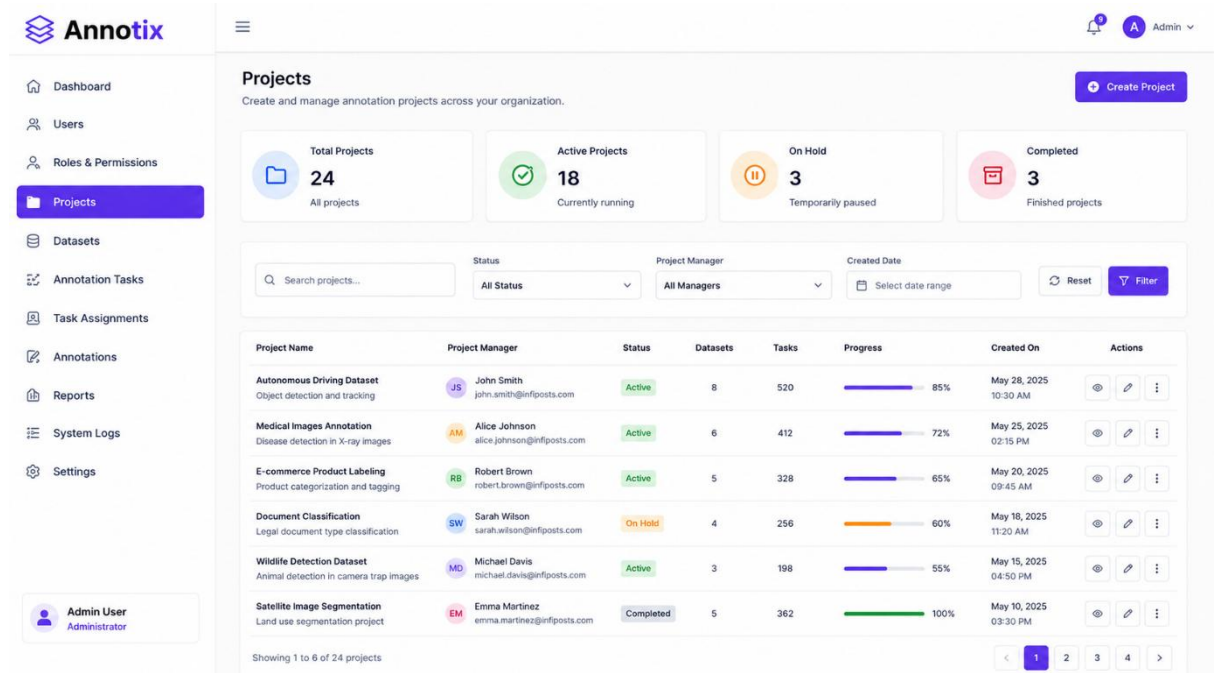
Displays the login interface through which users authenticate using their registered credentials. Upon successful authentication, a signed JWT is issued to the client for use in subsequent API requests.

Screenshot 5.2 Admin Dashboard Overview



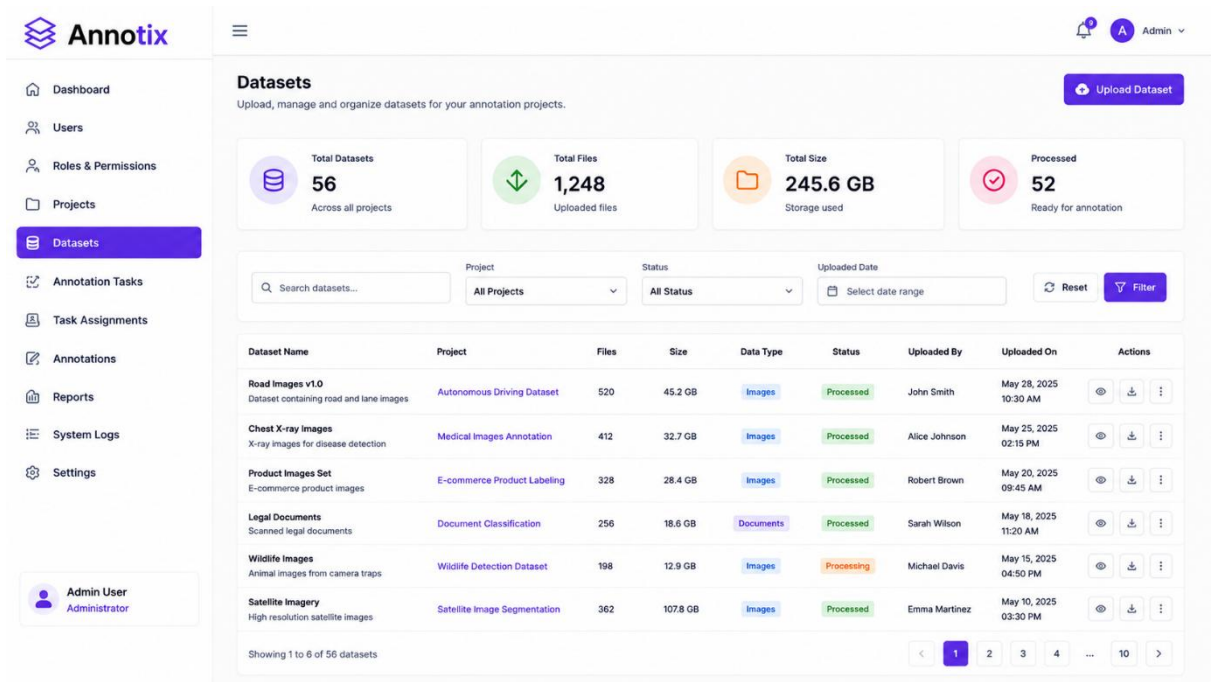
Shows the administrator dashboard, providing a summary view of active projects, registered users and overall annotation progress across the platform.

Screenshot 5.3 Project Creation and Management Screen



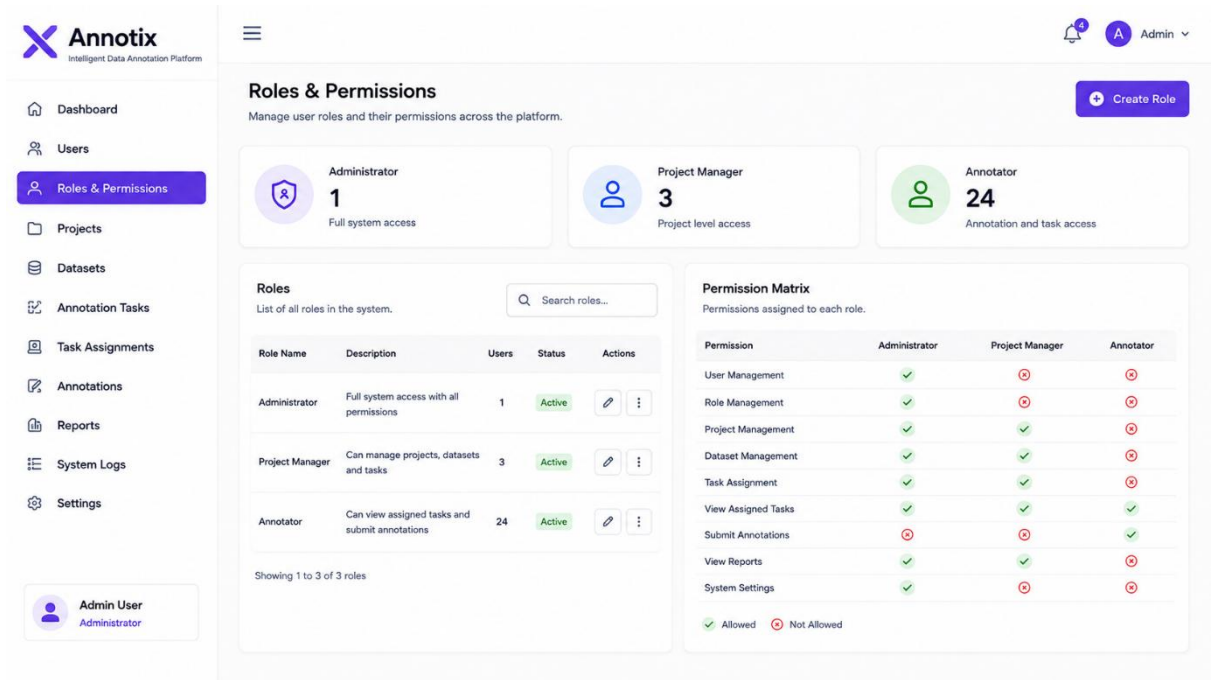
Illustrates the interface used by Project Managers to create new annotation projects, define project metadata, and associate datasets with a project.

Screenshot 5.5 Annotation Task Assignment Screen



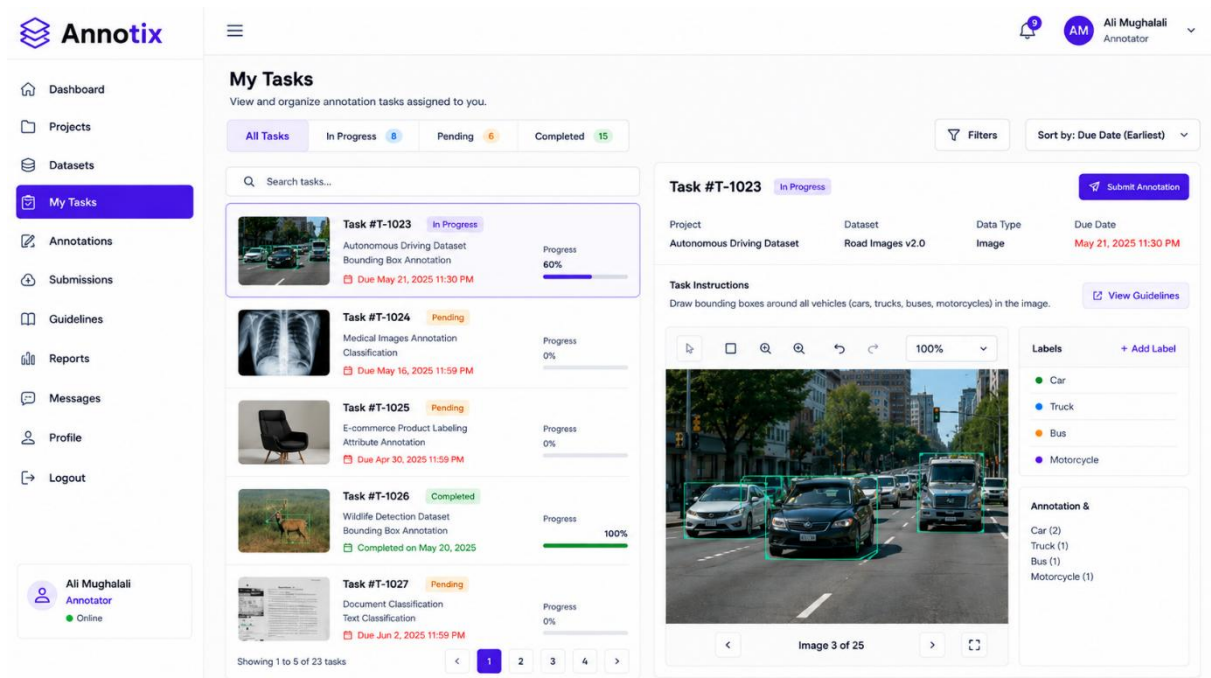
Shows the interface used to assign annotation tasks to individual annotators and to monitor the status of assigned tasks..

Screenshot 5.6 Role-Based Access Control (Role Management) Screen



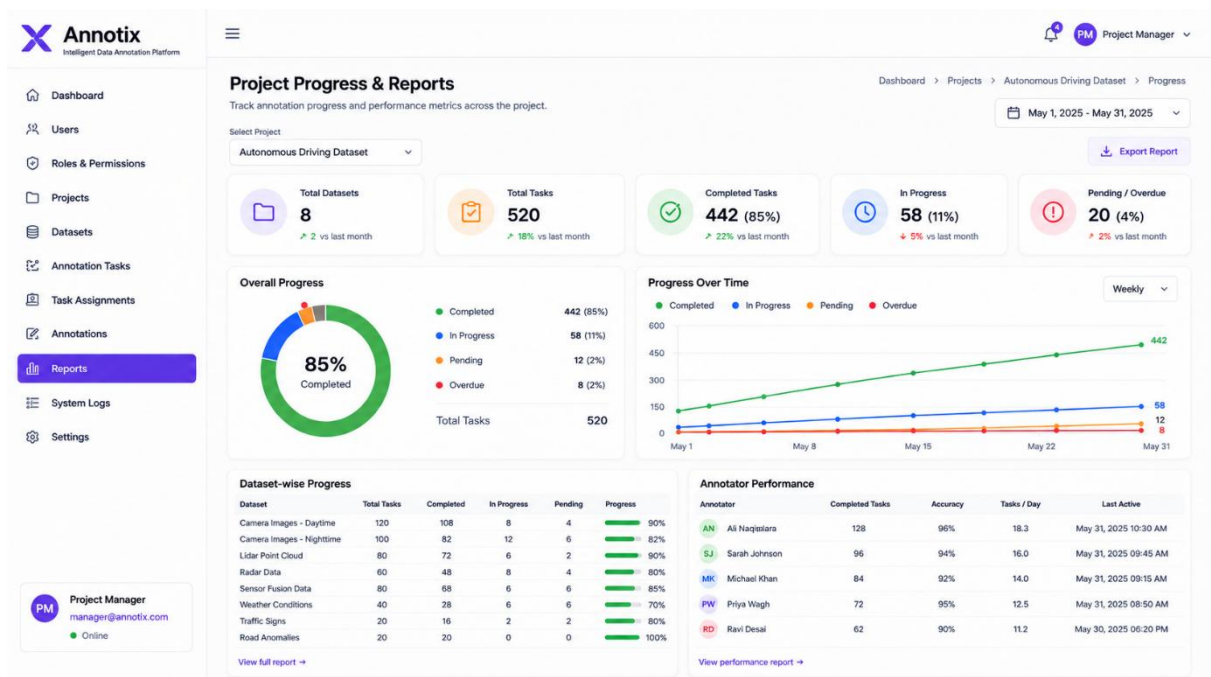
Illustrates the administrative screen used to manage user roles and permissions, reflecting the RBAC model implemented in the backend.

Screenshot 5.7 Annotator Task View



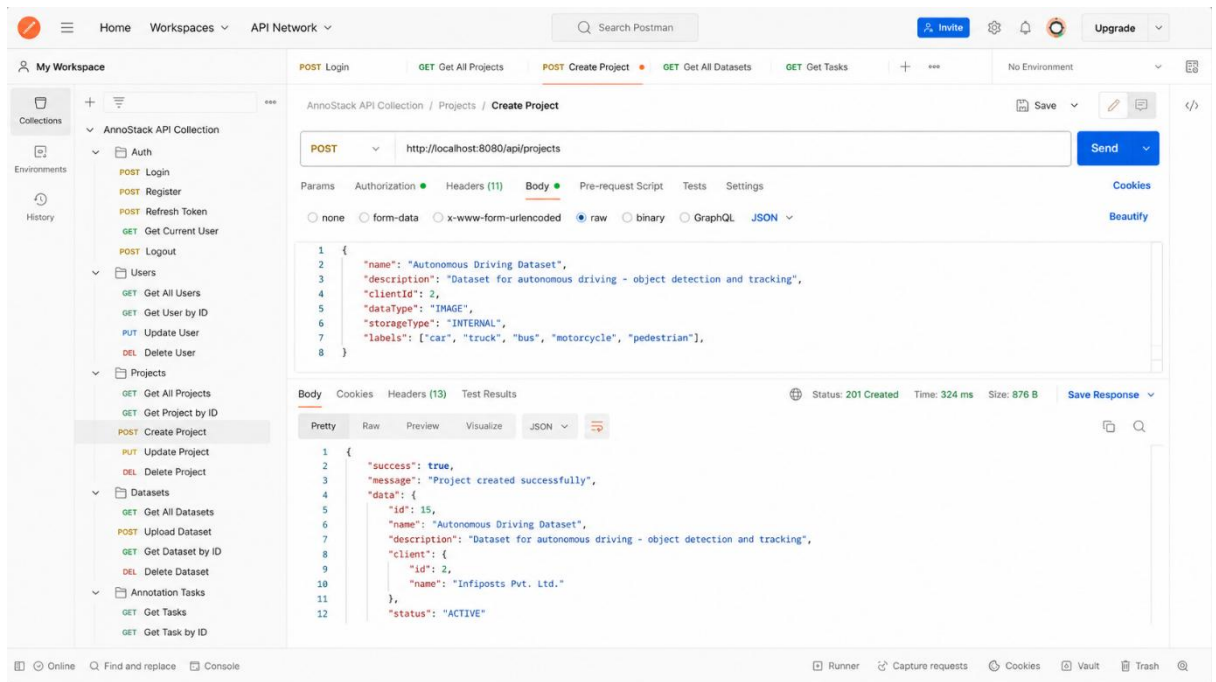
Displays the task list as seen by a user in the Annotator role, showing only the tasks assigned to that user in accordance with RBAC restrictions.

Screenshot 5.8 Progress Tracking and Reporting View



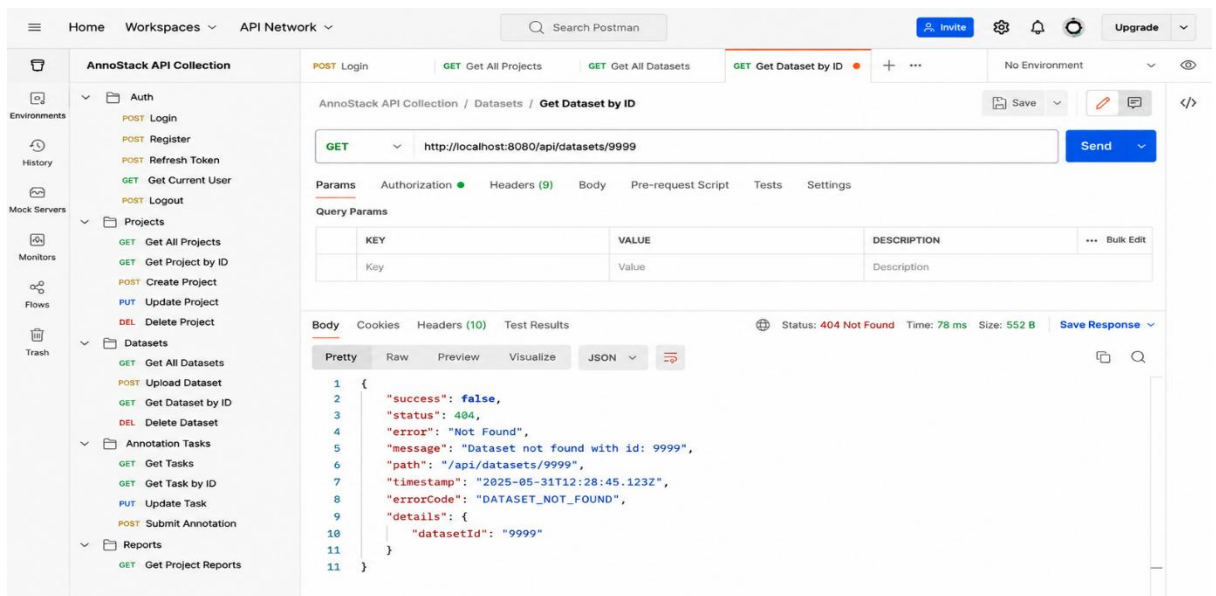
Shows a summary view of annotation progress across a project, including completed, in-progress and pending task counts.

Screenshot 5.9 Postman API Testing Collection



Illustrates a representative Postman collection used to test Annotix REST API endpoints, including authenticated requests bearing a JWT bearer token.

Screenshot 5.10 API Error Response Example



Shows a structured error response returned by the global exception handler in response to an invalid request, illustrating the consistent error-handling format adopted across the API.